Architecture applicative et choix techniques

Analyse du MVP de démarrage

L’application permet de sélectionner une commune et une culture afin d’étudier l’évolution d’un indicateur climatique. Le MVP calcule le nombre moyen annuel de jours de gel sur deux périodes, 2011-2020 et 2041-2050, à partir du modèle climatique MRI_AGCM3_2_S fourni par Open-Meteo.

Ce document décrit les choix effectivement présents dans le kit de démarrage. Les évolutions proposées en dernière partie sont distinguées des fonctionnalités déjà implémentées.

1. Architecture générale

Le MVP repose sur une **architecture client-serveur monolithique avec rendu côté serveur** (SSR, Server-Side Rendering). Le navigateur constitue le client : il présente les formulaires et affiche les résultats. Le serveur Flask reçoit les requêtes, orchestre les traitements et génère les pages HTML à partir de templates Jinja2.

L’interface et les traitements sont réunis dans un même projet applicatif. Ce choix réduit le nombre de composants à configurer et facilite le démarrage du prototype. Open-Meteo reste un service externe, interrogé par le backend ; le navigateur n’appelle pas directement cette API.

Composant	Responsabilité
Navigateur • HTML/CSS	Afficher le formulaire, transmettre les choix et présenter les résultats.
Serveur • Python/Flask/Jinja2	Recevoir la demande, obtenir les séries climatiques, calculer les indicateurs et produire le HTML.
Stockage • SQLite et fichiers	Conserver les indicateurs et référentiels ; mettre en cache les réponses API au format JSON.
Service externe • Open-Meteo	Fournir les séries de températures nécessaires aux calculs.

Périmètre : kit « mvp-demarrage » fourni pour la SAÉ 3.01.

2. Circulation des données

1. Sélection et requête
L'utilisateur choisit une commune et une culture. Le formulaire transmet ces paramètres au serveur par une requête HTTP GET vers la route /resultat.

2. Récupération des séries
Le serveur lit les caractéristiques de la commune et de la culture dans SQLite. Le client Open-Meteo recherche les températures minimales pour les deux périodes. Une réponse déjà mise en cache est réutilisée ; sinon, une requête est envoyée à l'API.

3. Calcul et enregistrement
Les séries sont chargées dans une table SQL temporaire. Le calcul compte, pour chaque année, les jours où la température minimale est inférieure au seuil de gel de la culture. Python calcule ensuite la moyenne annuelle par période et enregistre les résultats dans la table de faits.

4. Construction de la réponse
Flask relit les résultats enregistrés, prépare les données d'affichage et génère la page avec Jinja2. Le navigateur reçoit une page HTML complète, comprenant les comparaisons et les graphiques en CSS.

3. Organisation du code et relation avec MVC

Le projet suit une séparation partielle des responsabilités : **templates/** contient les vues, **openmeteo_client.py** gère les appels API et leur cache, et **data_init.py** initialise le schéma et les référentiels. Le fichier **app.py** regroupe les routes, l'orchestration des traitements, une partie des calculs et les requêtes SQL.

Cette organisation se rapproche d'un **MVC simplifié** : les routes remplissent le rôle des contrôleurs, les templates celui des vues, et les données ainsi que les règles de calcul correspondent au modèle. Il ne s'agit toutefois pas d'un MVC strict, car le modèle n'est pas isolé dans une couche dédiée. MVC décrit l'organisation interne du code ; client-serveur décrit la communication entre le navigateur et le serveur. Les deux approches sont compatibles.

4. Stockage et isolation des visiteurs

SQLite utilise un **schéma en étoile**. La table centrale **fact_indicateur** contient les résultats, reliés à quatre dimensions : commune, culture, temps et modèle climatique. Cette structure permet d'analyser un indicateur selon plusieurs axes et de préparer l'ajout de nouvelles comparaisons.

Chaque résultat est associé à un identifiant de session, conservé dans un cookie signé par Flask. Les lectures et écritures sont filtrées sur cet identifiant. Le bouton d'effacement supprime les résultats du visiteur puis réinitialise sa session. Le MVP ne comporte pas de comptes utilisateurs.

Les séries brutes ne sont pas conservées dans les tables permanentes de SQLite. En revanche, les réponses de l'API sont conservées sur disque dans le cache JSON, afin d'éviter des appels identiques.

5. Choix techniques et justifications

Choix	Justification dans le contexte du MVP
Python	Un même langage couvre le serveur, la récupération des séries et les calculs. Les modules standards suffisent pour HTTP, JSON, CSV et SQLite.
Flask ≥ 3.0	Le framework fournit les routes, les requêtes et les sessions avec peu de configuration. Il convient au périmètre réduit du prototype.
Jinja2 + HTML/CSS	Les pages sont générées sur le serveur. Les formulaires, tableaux et graphiques simples sont réalisables sans JavaScript ni framework frontend.
SQLite + SQL	La base fonctionne dans un fichier local, sans serveur supplémentaire. SQL permet les jointures et le calcul du nombre de jours de gel.
Schéma en étoile	La séparation entre faits et dimensions structure les comparaisons par territoire, culture, période et modèle.
API Open-Meteo	Elle fournit les séries climatiques nécessaires, ce qui permet de concentrer le développement sur leur traitement et leur restitution.
Cache JSON	La réutilisation des réponses réduit le temps d’attente et la consommation du quota API. Le client prévoit aussi des tentatives supplémentaires après une erreur 429.
Référentiels CSV	Les communes et les cultures peuvent être complétées dans des fichiers simples, puis importées dans les dimensions SQLite.

6. Limites et perspectives d’évolution

La priorité du MVP est la simplicité. Pour accompagner son enrichissement, les routes, la logique métier et les accès SQL pourront être séparés en modules dédiés. Cette évolution facilitera la maintenance et les tests sans imposer de séparer le frontend et le backend en deux applications.

Avant une mise en production, il faudra remplacer la clé de session de démonstration par un secret de configuration, désactiver le mode debug et adapter le déploiement. Une politique de purge des sessions abandonnées et du cache devra être définie. Des comptes utilisateurs permettront ensuite de retrouver ses résultats sur plusieurs appareils.

SQLite pourra être conservé tant qu’il répond aux besoins. Une migration vers une base serveur pourra être étudiée si le déploiement ou le volume d’écritures simultanées l’exige. Enfin, le document devra être actualisé si l’équipe modifie les technologies ou l’architecture du kit.

Base documentaire : README.md, requirements.txt, app.py, openmeteo_client.py et templates du kit fourni. Les pistes d’évolution ne sont pas présentées comme des fonctionnalités existantes.